System Design — Night-Before Cheat Sheet

*Condensed from STUDY_GUIDE.md. The one idea: everything is a trade-off — say it out loud.***

The framework (drive the conversation)

Scope → Estimate → API → Data model → Diagram → Deep-dive → Bottlenecks → Scale → Trade-offs 1. **Scope** — use cases (pick 2–3), users (DAU), QPS, **read:write ratio**, data size, latency/consistency needs. *Don't skip this — jumping to a solution is the #1 failure.* 2. **Estimate** — period→seconds (day ≈ 10^5 s); avg QPS = count/86,400; peak ≈ 2×; reads = writes × ratio; storage = count×bytes×retention; bandwidth = QPS×payload. 3. **API** — key endpoints (verb, path, params, response). 4. **Data model** — SQL vs NoSQL *and say why*. 5. **Diagram** — Client → DNS → CDN → LB → Web → App/Services → Cache ⇄ DB; slow work → Queue → Workers. 6. **Deep-dive** the core trick; **identify bottlenecks**; **scale each tier**; **name every trade-off**.

Six core trade-offs

- **Performance vs scalability** — slow for 1 user = perf; slow only under load = scalability.
- **Latency vs throughput** — maximize throughput at *acceptable* latency.
- **CAP** — partitions are inevitable (P not optional); real choice is **C vs A during a partition**. CP = err to stay correct (banking); AP = serve stale, reconcile later (feeds).
- **PACELC** — else (no partition): **latency vs consistency**. Cassandra/Dynamo = PA/EL; RDBMS/Spanner = PC/EC.
- **Consistency** — weak (games/VoIP) · eventual (feeds, DNS, most NoSQL) · strong (transactions, RDBMS).
- **Availability** — series multiplies (worse); parallel `1-(1-a)(1-b)` (better). 99.9%≈8.7h/yr, 99.99%≈52m/yr, 99.999%≈5m/yr.

Scaling toolbox (in order you reach for it)

1. **Vertical scale** (bigger box) — simplest, caps out, SPOF.
2. **Horizontal + load balancer** — needs **stateless** tier (sessions in Redis/DB). L4 = IP/port; L7 = URL/headers/cookies.
3. **Cache** (cache-aside = default) — biggest read win. Also write-through (never stale, slow writes), write-behind (fast, risk loss).
4. **CDN** — static/media near users (push = low traffic; pull = high traffic).
5. **Read replicas** (master-slave) — scale reads.
6. **Federation** (split DBs by function) → **Sharding** (split rows; consistent hashing) → **Denormalization** (kill joins).
7. **NoSQL** for the right shape: KV (Redis) · document (Mongo) · wide-column (Cassandra) · graph (Neo4j). BASE = availability over consistency.
8. **Async queues + workers** (Kafka/SQS/RabbitMQ) — decouple slow/spiky work; back pressure → 429 + backoff.

Key numbers

Memory ref **100 ns** · SSD read **150 µs** · same-DC round trip **0.5 ms** · disk seek **10 ms** · CA↔Netherlands **150 ms**. Memory ≈ 100,000× faster than disk seek. Seq read: disk 30 MB/s · 1 Gbps net 100 MB/s · SSD 1 GB/s · memory 4 GB/s. 2^10≈1K, 2^20≈1M, 2^30≈1B, 2^40≈1T. **1M/day ≈ 12 QPS.**

Capacity (one node, ±1 order of magnitude — for "one box vs many"): SQL writes **~1K/s** (>5K → shard/queue) · SQL reads ~10K/s (replicas) · Redis/Memcached ~100K ops/s · Cassandra ~10K writes/s **per node** (scales linearly) · Kafka 100K–1M msgs/s · app server 1K–10K QPS. **Memorize: single SQL primary ≈ 10^3 writes/s** — the Mint trigger for sharding.

Protocols

- **TCP** = reliable, ordered, slower (web/DB/SSH). **UDP** = lossy, fast, broadcast (VoIP/video/games).
- **REST** = public/CRUD, cacheable · **gRPC** = internal service-to-service, HTTP/2+Protobuf · **GraphQL** = client picks fields (mobile).
- Real-time: **SSE** = one-way push (feeds, LLM streaming) · **WebSockets** = full-duplex (chat/games) · long polling = fallback.
- HTTP verbs idempotent: GET/PUT/DELETE yes; POST/PATCH no. *Idempotent = safe to retry.*

Deeper topics (one-liners)

- **Consistent hashing** — ring of nodes+keys; node change moves only ~K/N keys; vnodes even out load.
- **B-tree** (read/range, RDBMS default) vs **LSM-tree** (write-optimized sequential SSTables + bloom filters; Cassandra).
- **Rate limit** — token bucket (bursts) / sliding-window counter; centralize in Redis (atomic Lua); return 429.
- **Idempotency** — exactly-once is impossible; do at-least-once + **idempotency keys** = effectively-once (payments, queue consumers).
- **Quorum** — **W + R > N** ⇒ strong consistency (N=3,W=2,R=2). Strong+fault-tolerant config → Raft store (etcd/ZooKeeper).
- **Bloom filter** — "definitely not / maybe present," no false negatives; skip expensive lookups.
- **Resilience** — timeout + retry(backoff+jitter, idempotent) + circuit breaker + bulkhead + graceful degradation. No SPOFs.
- **Observability** — logs (why) · metrics (that + alerts) · traces (where). SLI < SLO (error budget) < SLA.

When X → reach for Y

Symptom	Reach for
Read-heavy / hot keys	Cache + CDN, read replicas
Write-heavy ingest	LSM store, sharding, queue + batch
Expensive joins	Denormalize, materialized views
Slow inline work	Async queue + workers
Node churn reshuffles data	Consistent hashing
Duplicate messages	Idempotency keys / dedup
One slow dep stalls all	Timeout + circuit breaker + bulkhead
Traffic spikes	Autoscaling (stateless) + back pressure
Global agreement/config	Raft store (etcd/ZooKeeper)
Abuse / quotas	Token bucket (Redis)

Trade-off phrases to say out loud

- "Read-heavy (100:1), so I'll optimize reads with caching + replicas."
- "Stale feed is fine → I'll take AP / eventual consistency for availability."
- "Statelessness lets me autoscale the web tier (sessions in the cache)."
- "Fan-out on write makes reads cheap but breaks for celebrities → hybrid (pull for high-follower)."
- "I'll denormalize to avoid a cross-shard join, accepting write amplification."

- "That's a single point of failure — I'd add a standby / run it in parallel."
- "Benchmark → profile → fix the bottleneck → repeat. I won't jump to the final design."

Classic problems — the one insight

- **URL shortener** — `base62(md5(ip+time))[:7]`; metadata in SQL, blob in object store; analytics offline (MapReduce).
- **Twitter** — fan-out on write (precompute timelines, O(1) reads); hybrid for celebrities.
- **Web crawler** — content **signatures** break cycles / dedup; own DNS cache.
- **Mint** — async extraction via queue; store overrides not full templates; aggregate via MapReduce.
- **Social graph** — distributed **bidirectional BFS** over sharded graph; batch same-server lookups; shard by geography.
- **Search cache** — LRU = doubly-linked list + hash map (O(1)); shard via consistent hashing.
- **Sales rank** — batch problem → MapReduce; 2nd stage uses shuffle/sort for a distributed sort.
- **Scale on AWS** — iterate: single box → split DB → LB + stateless web → cache + replicas → autoscale → shard/NoSQL/async.